ROUND 2 · THE BUILD · KNOWLEDGE HANDOVER CHALLENGE

ContinuityOS

Your team's knowledge should never leave with a person.

Round 1 working name: Baton

Team	TEAM BATON (solo)
Author / Team Leader	Shaik Asad Ahmed · B.Tech CSE (AI), GPCET Kurnool
Working project link	https://wcc-hackathon-submission.onrender.com
Source repository	https://github.com/shaikasadahmed2k23/WCC_Hackathon_Submission
Demo accounts	Meera Iyer (approver) · Nisha Rao (restricted volunteer) · more on the login page. Password: continuity123
Data	All demo data is synthetic (fictional student-club app, CampusConnect)

Where the build stands. The core workflow runs end to end: connect sources, reconstruct, verify, review and approve, then share a role-filtered handover that a recipient acknowledges. Permission checks are enforced on the server and covered by automated tests. The default extractor is rule-based; the language-model extractor exists but has only been tested against a mocked model. Section 8 lists every limitation.

1. What changed since Round 1

The Round 1 plan promised a focused workflow rather than a long feature list. Round 2 delivers that workflow working end to end, and pulls forward several items that were scheduled for Round 3. Where we deviated from the plan, the reason is stated.

Round 1 plan	Round 2 reality	Why
Product name Baton	Renamed **ContinuityOS**	Clearer product identity; same concept and claim-ledger design
Claim ledger with quote, label, status, versions	**Delivered** as designed: claims carry evidence quotes, 8 labels, review status, versioned frozen snapshots with diff	Core of the plan
Quote-or-nothing verifier	**Delivered**: deterministic quote check plus a statement-support check; failures stored as Unverified	Core of the plan
Connectors: GitHub, notes, task CSV, chat export	**Delivered**: file upload (txt/md/csv/tsv/json), WhatsApp-style chat parser, pasted note, public GitHub repo import	As planned. PDF/DOCX parsing deferred
Redaction with Microsoft Presidio	Pattern-based redaction (keys, tokens, JWT, passwords, Aadhaar, PAN, card numbers with Luhn check), applied before storage	Presidio adds heavy install and model downloads; patterns cover the demo risks. Presidio stays on the roadmap
Supabase Postgres with Row-Level Security	SQLite with permission checks in the API layer, plus tests that assert no restricted text leaves the server	Judges can run it with zero setup. RLS comes with the Postgres migration
pgvector / Qdrant retrieval	Not used yet	Rule-based conflict detection was enough for the core workflow; semantic retrieval is a Round 3 improvement
Groq LLM extraction	Optional, off by default; any OpenAI-compatible endpoint; always verified	Keeps the demo reproducible and free. Tested with a mocked model only
Gap Radar + Last-mile Interview (Round 3)	**Gaps are detected and can be answered now**; the guided interview is still Round 3	Gap detection was needed for the Missing label
Claim-level permission engine + access requests (Round 3)	**Delivered now**: per-passage access levels, claim clearance, role preview, access requests and grants	Needed for the trust requirements, so it was pulled forward
Continuation plan + comprehension check (Round 3)	Plan and acknowledgement **delivered**; comprehension check moves to Round 3	Plan is the handover's end goal; the check needs interview data
React + Vite + Tailwind	React + Vite with hand-written CSS (Inter font)	Smaller bundle, consistent design tokens

2. Completed features

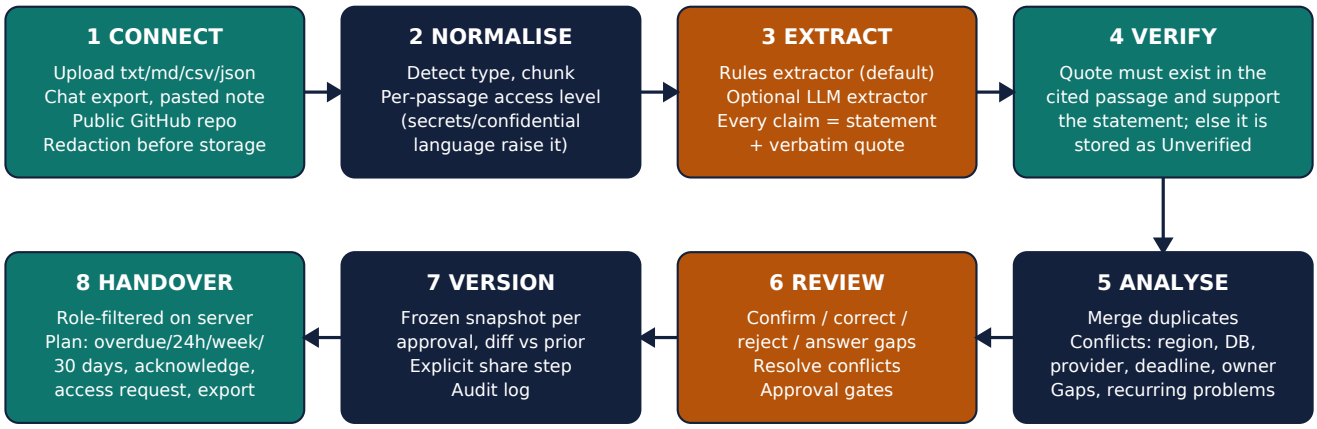
Workflow step	Feature	Status
Connect	Upload files, auto-detect task board / chat / document; paste a note; import a public GitHub repo; duplicate and binary-file rejection; size limit	Working
Connect	Per-source sensitivity (General / Internal / Confidential) and automatic raising of individual passages	Working
Reconstruct	Extraction of decisions, ownership, actions, risks, dependencies, project state, open questions, contacts; recurring-problem flag; de-duplication across sources	Working (rules)
Reconstruct	Language-model extraction through any OpenAI-compatible API	Built, mock-tested only
Verify	Verbatim-quote verifier; Unverified bin for unsupported model output; evidence panel with highlighted passage and neighbouring context	Working

Workflow step	Feature	Status
Verify	Conflict detection (deployment region, database, hosting, payment provider, deadlines, owners) with both sources and dates	Working
Verify	Gap detection: no owner, no recorded reason for a decision, tool mentioned only in chat, open questions	Working
Review	Confirm, correct (original preserved), reject, restore; bulk confirm of supported claims; resolve or accept-as-unresolved conflicts; answer gaps with attribution	Working
Review	Review decisions survive re-running reconstruction	Working, tested
Review	Readiness score with its arithmetic shown; approval blocked by open conflicts; explicit acceptance needed to approve unreviewed claims	Working, tested
Handover	Immutable versions with diff, explicit share step, role-filtered handover, continuation plan, acknowledgement of receipt	Working
Handover	Preview-as-role for approvers; access requests and grants; Markdown and JSON export (filtered the same way)	Working, tested
Trust	Append-only audit log; demo-grade authentication with hashed passwords; session expiry	Working
Usability	Five-step overview, label legend, empty and error states, keyboard-closable panels, responsive layout	Working

Demo workspace. The synthetic CampusConnect project loads 6 sources (43 passages) and produces 41 claims: 9 actions, 8 risks, 7 decisions, 7 gaps, 3 dependencies, 3 ownership, 2 project-state, 1 question, 1 contact. It deliberately contains two conflicts (API region: README says Mumbai, a later chat says Singapore; a webhook deadline of 15 Nov versus 8 Nov), an ownerless domain renewal, decisions with no recorded reason, a tool documented only in chat, a masked password, and a confidential budget message.

3. Architecture and data flow

Data flow of one reconstruction run



FastAPI (REST, bearer sessions, role checks on every route) · SQLite (claims, evidence, versions, audit)

React + Vite single-page app, served by the same FastAPI process in production

Source text is only ever treated as data; secrets never reach the database or a model

3.1 How a claim earns its label

Label	How it is produced	Where it can appear
Verified	Quote verified in the source, then confirmed or corrected by a reviewer	Counts as organisational fact
Supported	Quote verified in the source; no human has reviewed it	Shown with that wording; never called verified
Conflicting	Another claim disagrees on the same attribute	Blocks approval until resolved or accepted as unresolved

Label	How it is produced	Where it can appear
Missing	A gap the sources cannot answer	Shown as an open question with owner unknown
Stated	A person answered a gap	Attributed and dated, with the note that no document backs it
Inferred	Model reasoning across passages (supported by the data model, not produced by the rules extractor)	Dashed; cannot become Verified without a reviewer
Unverified / Rejected	Model quote not found in the source / reviewer removed it	Hidden by default; never in a handover

3.2 Why verification is deterministic

The verifier does not ask another model whether a claim is right. It checks that the quoted text exists in the cited passage (whitespace and case tolerant), and that at least 40% of the statement's content words appear in the quote. This makes the rule inspectable and means a model that invents or misquotes cannot produce a Supported claim. A test feeds a fake model an invented quote and asserts it is rejected.

4. Permissions and privacy

Role	Access level	Can do
Project admin	Confidential	Everything, including deleting sources and managing access
Approver	Confidential	Review, resolve, approve, share, preview as another role, decide access requests
Departing member	Confidential	Add sources, run reconstruction, answer gaps, read the draft. Cannot approve or confirm
Incoming (peer)	Internal	Read the shared handover, acknowledge, request access
Incoming (restricted)	General	Same, with the least visible content

- **Claim-level access.** Each passage has an access level (the source's level, raised automatically for secrets, confidential language, or contact details). A claim takes the highest level of its evidence.
- **Server-side withholding.** The handover endpoint, evidence context and both export formats filter on the server. Hidden items are reported only as counts by type. Tests assert that confidential strings are absent from the restricted roles' JSON and exports.
- **Secrets never stored.** Redaction runs before storage and before any model call. A test scans the database file and write-ahead log for the demo password.
- **Access requests.** A recipient can ask for one level more for one handover; an approver decides. Even a granted internal level never reveals confidential items (tested).
- **Prompt-injection stance.** Source text is data. The rules extractor ignores instructions; the LLM prompt marks excerpts as untrusted, the model has no tools, and its output goes through the verifier. A sample file with injected instructions is included and tested.

5. Technology, APIs, models and data

Layer	What we used
Frontend	React 18, Vite 6, lucide-react icons, @fontsource/inter; hand-written CSS with design tokens
Backend	Python 3.12, FastAPI, Pydantic, Uvicorn, python-multipart; SQLite via the standard library
AI	None required. Optional language-model extraction through any OpenAI-compatible chat-completions API (default endpoint and model: Groq, llama-3.3-70b-versatile). Not exercised against a live endpoint in development
External APIs	GitHub REST API for public repository import (unauthenticated, or GITHUB_TOKEN). Exercised through mocks; a live call from the build environment was rate-limited
Data	Synthetic CampusConnect sources written for this project (README, architecture note, two meeting notes, task board CSV, chat export) and two sample upload files
Tests	pytest (15 API tests), vitest + jsdom (4 UI tests against a running backend)

Layer	What we used
Deployment	Live on Render (free instance, Singapore region) from a single Docker image that builds the frontend and serves it from FastAPI; auto-deploys on every push to the public GitHub repository. Run scripts for Windows and Unix are included for local use
AI assistance	Claude (Anthropic) was used to design and write the code and this document

Reuse disclosure. A starter frontend zip supplied by the team informed the visual direction (navy sidebar, teal accent). Its code was not reused: it was a single file with hard-coded statistics and was rewritten as structured components wired to the real API.

6. Testing and evidence

Area	What is asserted
Authentication	Anonymous requests are refused; wrong passwords fail; staff-only routes return 403 for incoming roles; a departing member cannot confirm claims
No leakage	Restricted and peer handover JSON never contain the budget message or the password; contact details appear for peers only; exports follow the same rules; unshared users get 404
Approval gates	Approval is refused while conflicts are open and while unreviewed claims exist unless explicitly accepted
Review integrity	Verified claims and edits survive re-running reconstruction
Verifier	A fake model's invented quote is rejected; a real quote is accepted; unknown chunk ids are dropped
Ingestion failures	Binary files, empty files and duplicate uploads are rejected with clear messages; a file containing injected instructions yields no Verified claim
Secrets	A password in the chat export is not present anywhere in the database files
Access flow	Request, approve, view with raised access, still no confidential content; acknowledgement works only for recipients
UI journeys	Login, overview, evidence highlight, conflicts tab, blocked approval, restricted recipient view, acknowledgement, access request, role preview

Not yet tested: live language-model extraction, live GitHub import, behaviour with large repositories, concurrent editing by several reviewers, and accessibility with a screen reader (keyboard closing and labelled controls are in place).

7. Three major improvements committed for Round 3

#	Improvement	What it adds beyond Round 2	How we will show it
1	**Last-mile Interview and Day-1 comprehension check**	A guided interview for the departing person built from the Gap Radar: each question shows the evidence that triggered it, accepts typed or dictated answers, asks one grounded follow-up, and stores answers as attributed claims that are checked against existing claims for contradictions. The incoming person then completes a short check on the highest-risk claims; scores feed readiness and the audit log.	Complete an interview on the demo project; show contradictions caught; show a recipient's comprehension result
2	**Live connectors with incremental sync and permission mapping**	OAuth or token-based connectors for private GitHub repositories, Slack channels, Jira projects and Google Drive. Sync cursors fetch only changes; changed passages re-open affected claims as needing re-review; source-system permissions map onto access levels instead of manual labelling.	Connect a real or sandbox account, edit a source, show the claim flagged for re-review and the access level mapped automatically
3	**Semantic evidence engine with measured accuracy**	Embedding-based retrieval and semantic conflict and duplicate detection beyond keyword lexicons, language-model extraction run live with calibrated confidence, and PDF/DOCX ingestion. A labelled benchmark (synthetic plus public repositories) reports precision and recall for claims, conflicts and gaps, and the report ships in the repository.	Publish the benchmark table; show cases the rules missed that the semantic engine catches, with the verifier still gating them

Beyond Round 3 (roadmap): Postgres with Row-Level Security, SSO and MFA, encryption at rest, Presidio-based redaction, policy packs for retention, multilingual handovers.

8. Known limitations

- The default extractor is pattern-based. It is accurate on explicit statements and misses implied knowledge, sarcasm and anything written in an unexpected style. The reviewer step exists because of this.
- Conflict detection covers a fixed set of attributes (region, database, hosting, payments, deadlines, owners). Other contradictions are not found yet.
- Redaction is pattern-based and will not catch every secret or personal detail. Treat connected sources as sensitive.
- The live site runs on a free Render instance: it sleeps after inactivity, so the first load can take up to a minute. The demo database resets on every restart or redeploy, so approved or shared handovers can disappear and the demo data reloads.
- Authentication is demo-grade (no SSO or MFA). SQLite data is not encrypted at rest.
- Passage-level access is automatic, but a claim that merges evidence from several sources takes the strictest level, so some information is hidden more than strictly necessary.
- Only text-based files are parsed. GitHub import is built but was only tested with a mocked API, and the language-model extractor is switched off in the live product. Large repositories are sampled (first 40 issues and pull requests).

Contribution. Shaik Asad Ahmed built everything in this submission alone: product design, backend, extraction and verification pipeline, permission model, frontend, synthetic dataset, tests and documentation.